BIKE-SHARING STATION REBALANCING OPTIMIZER

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

KOLLIPARA TEJA

Student ID: 2601050118

under the supervision of

Dr. RAJU ANITHA

Professor



Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd., u/s 3 of UGC Act 1956)

Green Fields, Vaddeswaram-522502, Guntur (Dist.), Andhra Pradesh, India

October 2026

KONERU LAKSHMAIAH EDUCATION FOUNDATION
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

I hereby declare that the project report entitled "Bike-Sharing Station Rebalancing Optimizer" is prepared for academic purposes under the guidance of Dr. RAJU ANITHA, Professor ,Department of Computer Science and Engineering. The report describes the design and implementation of a Python application that computes target bike inventories from predicted demand, plans truck-based rebalancing with a Greedy algorithm and a Dynamic Programming algorithm, and evaluates scenarios serially and in parallel.

I declare that the technical descriptions, algorithms, implementation details and evaluation discussion have been prepared to represent the project accurately. Sources consulted in preparing the report are acknowledged in the references. Behaviour that was not measured is described as design intent or future work and not as a completed experimental result. This report is submitted in accordance with the institution's academic-integrity requirements.

Place: Vaddeswaram

Signature of the Student: KOLLIPARA TEJA (2601050118)

ACKNOWLEDGEMENT

I express my sincere gratitude to Dr. RAJU ANITHA, Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, for the valuable guidance and suggestions given throughout the Bike-Sharing Station Rebalancing Optimizer project.

I thank the Department of Computer Science and Engineering and Koneru Lakshmaiah Education Foundation for providing the academic support and facilities needed to study optimization, greedy algorithms, dynamic programming and parallel computing.

I also acknowledge the researchers and authors whose work informed the concepts and algorithms discussed in this report. Finally, I thank everyone who encouraged and supported the completion of this academic project.

KOLLIPARA TEJA (2601050118)

INDEX

S. No.	Topics	Page No.
1	**Introduction and Problem Definition**	1 – 3
2	**Literature Review and Theoretical Foundations**	4 – 6
3	**Proposed Methodology and System Architecture**	7 – 12
4	**Implementation and Dashboard Design**	13 – 15
5	**Experimental Setup, Results and Discussion**	16 – 21
6	**Conclusion, Limitations and Future Scope**	22
7	**References**	24

LIST OF TABLES

No.	Table Name	Page
2.1	Comparison of related approaches	5
3.1	Software requirements	7
3.2	System architecture layers	9
3.3	Complexity of the implemented modules	12
4.1	Project file organization	13
4.2	Worked example: Greedy versus Dynamic Programming	14
4.3	Class responsibilities	13
4.4	Dashboard configuration parameters	15
5.1	Test environment and parameters	16
5.2	Example scenario: Greedy versus Dynamic Programming	17
5.3	Truck routes in the example scenario	17
5.4	Summary over 30 random scenarios	18
5.5	Optimization time versus number of stations	19
5.6	Effect of the truck-balancing option	20
5.7	Serial versus parallel scenario batches	20
5.8	Automated test suite	21

ABSTRACT

Bike-sharing systems suffer from a persistent operational problem: stations in busy areas run out of bikes while stations elsewhere fill up and have no free docks. Operators correct this by sending trucks to move bikes between stations, but the trucks are few, their capacity is limited, and every kilometre driven costs money. A rebalancing tool should therefore decide how many bikes each station should hold, which stations should give bikes and which should receive them, and how the trucks should carry out the transfers.

This project presents a Bike-Sharing Station Rebalancing Optimizer written in Python. A demand analyzer converts predicted demand into integer target inventories that respect station capacity and conserve the total number of bikes. Two optimizers then produce transfer plans on identical scenarios: a Greedy algorithm that serves the largest shortage from the nearest surplus station, and a Dynamic Programming algorithm that solves the underlying transportation problem exactly by memoised search over remaining station amounts. A route planner splits transfers into truck trips, assigns trips to a limited fleet, and executes the routes on a copy of the inventory so that every constraint is verified. Both plans are scored with the same objective, a weighted sum of truck distance, remaining imbalance and trucks used. A multiprocessing runner evaluates many independent scenarios concurrently, and a Streamlit dashboard presents the map, routes, inventories and comparisons.

On a 12-station example the Greedy and Dynamic Programming plans both removed the full initial imbalance of 62 bikes, while Dynamic Programming reduced transfer cost by 13.8% and truck distance by 12.8%. Over 30 seeded scenarios Dynamic Programming never had a higher transfer cost than Greedy (117.3 versus 127.8 bike-km on average), although its truck route was not always shorter because transfer cost is only a simplified objective. Dynamic Programming completed within its work budget only for smaller problems, whereas Greedy handled 50 stations in under one millisecond. On a two-core test machine the parallel runner gave a speed-up of 1.74× for 96 runs and was slower than a single process for 16 runs, confirming that parallelism pays off only for sufficiently large workloads. The system is an educational prototype using synthetic data and is not connected to live operational data.

Keywords: bike sharing, rebalancing, greedy algorithm, dynamic programming, transportation problem, route planning, multiprocessing, Streamlit.

1. INTRODUCTION

1.1 Background

Bike-sharing systems let people rent a bicycle at one station and return it at another [1]. Their use is rarely balanced. Commuter destinations, universities and transit hubs lose bikes in the morning and gain them in the evening, while residential stations show the opposite pattern. Over time some stations become empty, so riders cannot start a trip, and others become full, so riders cannot finish one [2]. Both situations reduce service quality and, for the operator, lost trips mean lost revenue and dissatisfied users.

Operators address this by *rebalancing*: a small fleet of trucks collects bikes from stations that have more than they need and delivers them to stations that have less [3]. Rebalancing during the night, when demand is low, is called static rebalancing, and it is the setting studied in this project [4], [5]. The planning task is to decide which stations send bikes, which stations receive them, how many bikes move between each pair, and how the trucks are routed so that the work is done with limited vehicles and as little driving as possible.

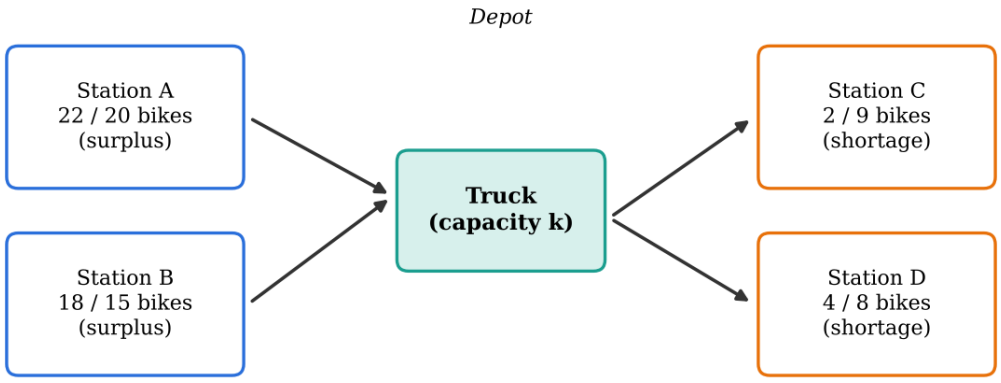


Figure 1.1: Bikes move from surplus stations to shortage stations by truck

A common way to define a good inventory is to compare each station's current bike count with a *target* derived from predicted demand. Stations above their target have a surplus, stations below it have a shortage, and the sum of absolute differences is the total imbalance. The goal of rebalancing is to reduce this imbalance towards zero while respecting the capacity of every station and every truck.

1.2 Problem Definition

Given a set of stations with locations, capacities, current bike counts and predicted demand, a depot, and a fleet of trucks with a fixed carrying capacity, produce a rebalancing plan that (a) moves bikes so that each station approaches its target inventory, (b) never takes more bikes from a station than it has, never overfills a station and never overloads a truck, (c) uses no more trucks than are available, and (d) keeps truck travel low. Because truck distance, remaining imbalance and the number of trucks all matter, the plan must be judged by a single clearly defined objective so that different algorithms can be compared fairly.

The project addresses three connected problems. First, it must compute sensible target inventories from predicted demand. Second, it must generate transfer plans with a fast heuristic and with an exact method, and turn them into feasible truck routes. Third, it must measure solution quality and running time on the same scenarios, including how the methods behave as the problem grows and whether independent scenarios can be evaluated faster in parallel.

1.3 Project Objectives

- Identify the bike surplus and shortage of every station from current inventory and demand-based targets.
- Create valid pickup and delivery assignments that respect station inventory, station capacity, truck capacity and the number of available trucks.
- Implement a Greedy baseline and a Dynamic Programming optimizer behind a common interface.
- Reduce total imbalance and truck distance, using a documented multi-term objective score.
- Compare both algorithms on identical scenarios and report quality, distance and execution time.
- Use multiprocessing to run independent scenarios concurrently and measure the speed-up.
- Provide a Streamlit dashboard and a command-line interface, and verify the logic with automated tests.

1.4 Scope

The current implementation covers static, single-night rebalancing with synthetic or user-supplied station data. Each truck trip picks up bikes at one station and delivers them to one other station. The system does not collect live trip data, does not predict demand itself (predicted demand is an input), does not model traffic, time windows or driver shifts, and does not control any real vehicle. Distances are straight-line (great-circle) distances between station coordinates, not road distances.

1.5 Existing System

In a simple operation, rebalancing may be done by fixed rules (for example, always refill the busiest stations first) or by the experience of drivers. Such methods are easy to apply but do not quantify how much imbalance remains, how far the trucks will drive, or whether another plan would be cheaper. A purely greedy rule is fast and intuitive, yet it can make an early choice that forces expensive transfers later.

Exact optimization methods for rebalancing exist in the research literature [5], [6], but they are typically presented as mathematical models rather than as an inspectable tool in which a student can change the number of stations or trucks, see the routes on a map, and compare an exact method with a heuristic on the same data.

1.6 Proposed System

The proposed system is a modular Python application with six layers: a user interface, data management, a demand and imbalance analyzer, an optimization engine, route planning with parallel execution, and results visualization. The demand analyzer computes integer targets proportional to predicted demand and capped by station capacity. The optimization engine

contains a Greedy optimizer and a Dynamic Programming optimizer that both return a list of transfers. The route planner converts transfers into truck trips and validates them, and the simulation engine scores both plans with the same objective function.

The Greedy and Dynamic Programming optimizers are deliberately separated from the user interface so that they can be tested from the command line and by automated tests without starting the dashboard. This separation also makes it straightforward to connect the same logic to a web service later.

1.7 Advantages of the Proposed System

- Compares a fast heuristic and an exact method on identical inputs and identical scoring.
- Enforces all inventory, capacity and fleet constraints by executing every plan on a copy of the inventory.
- Shows routes on a map and lists each truck's pickups and deliveries.
- Keeps algorithms independent of the user interface, which supports unit testing.
- Uses seeded synthetic data, so every experiment can be repeated exactly.
- Runs independent scenarios in parallel and reports the measured speed-up honestly, including cases where parallelism does not help.

1.8 Applications

The concepts demonstrated here apply to bike and scooter sharing, car-sharing relocation, and any system in which a limited resource must be moved between locations to follow predicted demand, such as shared lockers, battery swapping or retail stock transfers. They are conceptual rather than claims of direct production use. A real operator would also need live demand forecasts, road distances, working-time rules and traffic information.

2. LITERATURE REVIEW AND THEORETICAL FOUNDATIONS

2.1 Introduction

Bike-sharing rebalancing connects several established areas: transportation and operations research, combinatorial optimization, dynamic programming, vehicle routing and parallel computing. This chapter summarizes the concepts used by the implementation and explains why the chosen methods suit a small, inspectable prototype. The project uses well-known approaches rather than proposing a new optimization result; its contribution is the integration of these methods into a tested tool that compares them on identical scenarios.

2.2 Bike-Sharing and the Rebalancing Problem

Bike-sharing programmes have spread rapidly across Europe, the Americas and Asia [2], and their operation raises a distinctive logistics problem [1], [3]. The *static rebalancing problem* asks for truck routes and loading and unloading quantities that bring station inventories to desired levels before demand resumes. Chemla et al. showed that this problem is NP-hard and proposed exact and heuristic solution approaches [5]. Raviv et al. studied static repositioning with models that account for user dissatisfaction and operating cost [4], and Dell'Amico et al. proposed mathematical formulations and benchmark instances [6]. These works combine station-to-station transfers with full vehicle routing, which is considerably harder than the transfer-selection subproblem studied here.

2.3 Targets from Predicted Demand

A target inventory expresses how many bikes a station should hold. A natural rule gives each station a share of the system's bikes in proportion to its predicted demand. Because a station cannot hold more than its capacity, a proportional share that exceeds capacity must be capped and the remainder redistributed among the other stations; this is the classical *water-filling* procedure. Finally, shares must be converted to integers without changing the total, which can be done by the largest-remainder method. The result is a set of targets T_i with $0 \le T_i \le C_i$ and $\Sigma\, T_i$ equal to the total number of bikes, so rebalancing can be performed without adding or removing any bike.

2.4 The Transportation Problem

Once targets are known, each station has either a surplus $s_i = \max(B_i - T_i, 0)$ or a shortage $n_j = \max(T_j - B_j, 0)$. Choosing how many bikes x_{ij} to send from surplus station i to shortage station j so that every shortage is met at minimum total cost $\Sigma\, d_{ij} \times x_{ij}$ is the *transportation problem* introduced by Hitchcock [7] and studied extensively in linear programming [8]. Its constraints are $\Sigma_j x_{ij} \le s_i$, $\Sigma_i x_{ij} = n_j$ and $x_{ij} \ge 0$ and integer. Specialised algorithms such as the Hungarian method relate to its special case, the assignment problem [9]. In this project the transportation problem is solved by dynamic programming because the method is easy to explain, exact for the integer model, and instructive about the growth of the state space.

2.5 Greedy Algorithms

A greedy algorithm builds a solution by repeatedly making the locally best choice without reconsidering earlier decisions [11]. For rebalancing, a natural greedy rule takes the station with the largest shortage and supplies it from the nearest station that still has surplus. This rule is fast,

uses little memory and is easy to implement, but it can be misled: using the nearest source for one shortage may remove the cheapest source of another shortage, forcing a long transfer later. Greedy algorithms are therefore a good baseline and a good choice when answers are needed quickly on large instances, but they offer no optimality guarantee for this problem.

2.6 Dynamic Programming

Dynamic programming solves a problem by combining optimal solutions of overlapping subproblems and storing each subproblem's answer so it is computed only once [10], [11]. It applies when the problem has *optimal substructure*: an optimal solution contains optimal solutions of its subproblems. For the transportation problem, after deciding how a given shortage station is served, the remaining decision is the same kind of problem with reduced supplies and one fewer shortage station. The state is therefore the vector of remaining supplies together with the current stage, and the value of a state is the minimum cost of serving all remaining shortages.

The cost of dynamic programming is the size of the state space. If the supplies are $s_1, \ldots, s_m$, the number of distinct remaining-supply vectors can reach the product of $(s_i + 1)$. This grows exponentially with the number of stations on the state side, although it is polynomial in the number of bikes for a fixed number of stations. It is for this reason that the implementation chooses the side with the smaller state space, enforces a work budget, and reports a run as skipped rather than running for an unreasonable time.

2.7 Routing, Distance and Objective Functions

The transportation cost $\Sigma\, d_{ij} \times x_{ij}$ counts bike-kilometres, not truck-kilometres. A truck that carries ten bikes over five kilometres incurs five truck-kilometres, not fifty, and a truck may visit several stations in one tour. Actual truck distance depends on how trips are sequenced and on how many trucks are used, which is the vehicle routing problem [12]. The project therefore separates the two concerns: the optimizers decide the transfers, a route planner builds truck routes, and a single objective score, $\alpha \times D + \beta \times I_{after} + \gamma \times V$, is used to compare plans. Station-to-station distances are computed from latitude and longitude with the haversine formula [13].

2.8 Serial and Parallel Computation

Independent scenarios can be evaluated concurrently on several processor cores. In Python, true parallelism for CPU-bound work uses separate processes because of the interpreter's global lock, and the standard library offers the multiprocessing module and concurrent futures process pools for this purpose [14], [15]. Parallel execution has costs: processes must be started, work must be sent to them, and results must be collected. Speed-up is defined as serial time divided by parallel time, and parallel efficiency is speed-up divided by the number of workers. When each task is small, these overheads can exceed the benefit.

2.9 Comparison of Related Approaches

Approach	Strength	Limitation	Role in this project
Manual / rule-based	Simple, needs no software	No measure of quality or cost	Motivation
Greedy heuristic	Very fast, scales to large systems	May miss cheaper combinations	Implemented baseline

Approach	Strength	Limitation	Role in this project
Dynamic programming	Exact for the stated transfer model	State space grows quickly	Implemented exact optimizer
Full routing models [5], [6]	Model whole tours and timing	Hard to solve and explain	Future extension
Multiprocessing	Faster for large independent workloads	Process overhead on small ones	Implemented parallel runner

Table 2.1 Comparison of related approaches

2.10 Summary

The theoretical basis of the system is demand-proportional targets with water-filling, the transportation problem for choosing transfers, greedy selection as a fast baseline, dynamic programming for exact optimization, and process-based parallelism for independent workloads. Each concept maps to a module that produces a verifiable result: targets that conserve bikes, transfers with a computed cost, routes that pass constraint checking, and timing and equality measurements. The model is intentionally small enough to explain and does not capture time-varying demand, user behaviour or live traffic; these limits are discussed in Chapter 6.

3. PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 Introduction

The methodology divides the application into algorithm modules and a presentation layer. Data management supplies stations, demand and distances. The demand analyzer computes targets and imbalance. Two optimizers produce transfers. The route planner converts transfers into truck routes and validates them. The simulation engine evaluates plans with a common objective, and the parallel runner executes many scenarios concurrently. The dashboard and the command-line interface are clients of these modules.

3.2 System Requirements

3.2.1 Hardware Requirements

- **Processor:** a modern laptop or desktop CPU; multiple cores are useful for the parallel scenario runner.
- **Memory:** enough RAM for Python, the browser dashboard and the dynamic-programming memo table (a few hundred megabytes are sufficient for the supported sizes).
- **Storage:** space for the source files, the Python virtual environment and exported results.
- **Display and input:** a screen able to show the map and tables, with keyboard and pointing device.

3.2.2 Software Requirements

Component	Technology	Purpose
Programming language	Python 3.10 or newer	Algorithms, simulation, object-oriented model
Numerical computing	NumPy [18]	Distance matrix, random scenario generation
Data handling	pandas [19]	Loading CSV data and result tables
Dashboard	Streamlit [16]	Interactive configuration, tables and charts
Charts	Plotly [17]	Station map, inventory and comparison charts
Parallel execution	multiprocessing / ProcessPoolExecutor [14], [15]	Concurrent scenario evaluation
Testing	pytest / unittest	Constraint, edge-case and algorithm tests

Table 3.1 Software requirements

3.3 Functional Requirements

FR-1 — Data input: the user can generate a seeded synthetic system, load the sample CSV files, or upload stations and demand files.

FR-2 — Target computation: the system computes integer target inventories proportional to predicted demand, within capacity, conserving the total bikes.

FR-3 — Imbalance analysis: the system classifies every station as surplus, shortage or balanced and reports the total imbalance.

FR-4 — Optimization: the user can run the Greedy optimizer, the Dynamic Programming optimizer, or both on the same scenario.

FR-5 — Truck routing: the system assigns transfers to a limited number of trucks with a given capacity and reports every pickup and delivery.

FR-6 — Evaluation: the system reports imbalance before and after, bikes moved, truck distance, trucks used, execution time and the objective score.

FR-7 — Parallel batch: the user can run many seeded scenarios on several worker processes and view the speed-up.

3.4 Non-Functional Requirements

Correctness: no plan may take more bikes than a station holds, overfill a station, overload a truck, use more trucks than available, or leave bikes on a truck. Total bikes must be conserved. Both algorithms must be evaluated on identical data with identical scoring.

Performance: the Greedy optimizer should respond immediately for interactive use. The Dynamic Programming optimizer should either finish in reasonable time or report that the problem is too large.

Reproducibility: results for the same seed and settings must be identical, including when computed by parallel worker processes.

Maintainability: algorithms are separated from the user interface, so they can be tested from the command line and reused by other front ends.

3.5 System Architecture

The architecture has six layers, as shown in Figure 3.1 and summarized in Table 3.2. Each layer has a focused responsibility, which makes the code easier to test and extend.

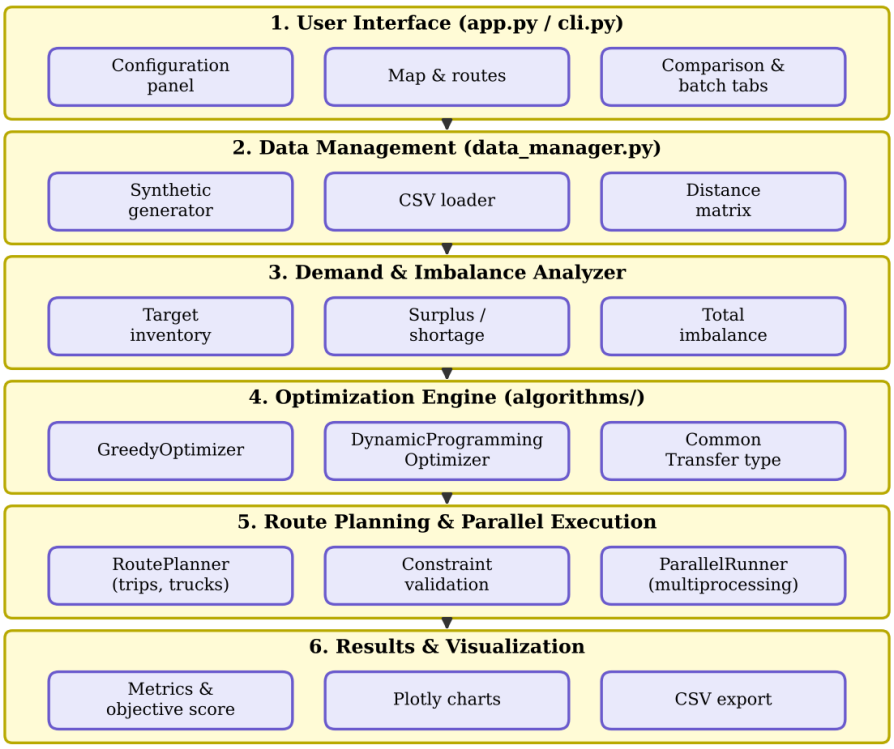


Figure 3.1: Logical architecture of the rebalancing optimizer

Layer	Main code	Responsibility
1. User interface	app.py, cli.py	Collect configuration; show results
2. Data management	services/data_manager.py	Generate or load stations and demand; build distance matrix
3. Demand & imbalance analyzer	services/demand_analyzer.py	Compute targets, surplus, shortage and total imbalance
4. Optimization engine	algorithms/	Produce transfers with Greedy or Dynamic Programming
5. Route planning & parallel execution	services/route_planner.py, parallel_runner.py	Build and validate truck routes; run scenarios concurrently
6. Results & visualization	services/simulation.py, visualization/charts.py	Compute metrics and objective; draw charts

Table 3.2 System architecture layers

3.6 Data Flow

A scenario starts as a table of stations (identifier, coordinates, capacity, current bikes) and a table of predicted demand. The data manager builds station objects, computes the great-circle distance matrix including the depot, and calls the demand analyzer to set every station's target. The simulation engine passes a copy of the system to the chosen optimizer, which returns a list of transfers (source station, destination station, number of bikes). The route planner turns the transfers into trips and assigns them to trucks. The same engine then executes the routes on the inventory, computes the final imbalance, and returns a result object that the dashboard or command line formats.

3.7 Target and Imbalance Computation

Let B_i be the current bikes and C_i the capacity of station i, and let w_i be its predicted demand. The total number of bikes is $N = \Sigma B_i$. Initially each station is allotted $N \times w_i / \Sigma w$. Any station whose allotment exceeds C_i is fixed at C_i and removed, and the remaining bikes are redistributed among the other stations in proportion to demand; this repeats until no station exceeds capacity. The fractional targets are rounded down and the remaining bikes are given, one at a time, to the stations with the largest fractional parts that still have room. The initial imbalance is $I_{before} = \Sigma |B_i - T_i|$.

3.8 Greedy Algorithm Workflow

The Greedy optimizer builds a surplus map and a shortage map. While both are non-empty, it selects the shortage station with the largest remaining need, finds the station with surplus that is nearest to it, transfers $q = \min(\text{surplus}, \text{need})$ bikes, updates both maps and removes entries that reach zero. Ties are broken by station index so that results are deterministic. Each iteration finishes at least one surplus or shortage entry, so the number of iterations is at most the number of surplus plus shortage stations.

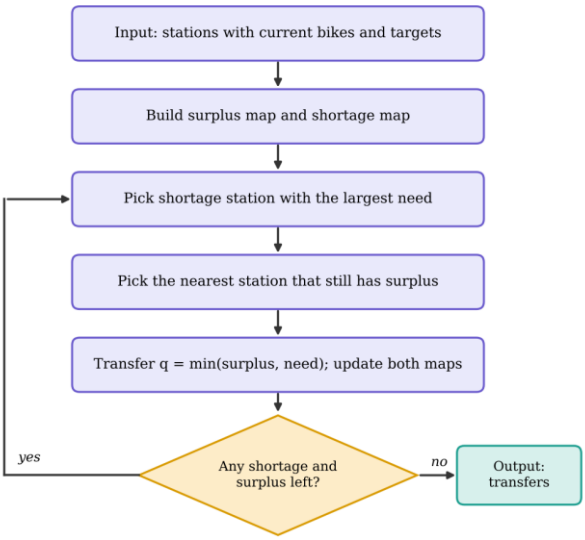


Figure 3.2: Greedy algorithm workflow

3.9 Dynamic Programming Algorithm Workflow

The Dynamic Programming optimizer solves the transportation problem exactly. It treats one side as the *stages* (for example, the shortage stations, processed one after another) and stores the remaining amounts of the other side in the *state*. Within a stage it decides, station by station, how many bikes q to send from the current station to the current stage, adding the cost $d \times q$ and the value of the resulting next state. A memo table stores the best value and the best q for every state, so each state is solved once. After the first state has been solved, the solution is read back by following the stored decisions.

Two refinements keep the method usable. First, the optimizer compares the sizes of the two possible state spaces, the product of (supply + 1) and the product of (need + 1), and uses the

smaller one as the state; since total supply equals total need after target computation, the problem is symmetric. Second, a work budget of three million transitions is enforced. If the budget (or Python's recursion depth) is exceeded, a ProblemTooLargeError is raised and the simulation engine reports the run as *skipped* instead of failing or hanging.

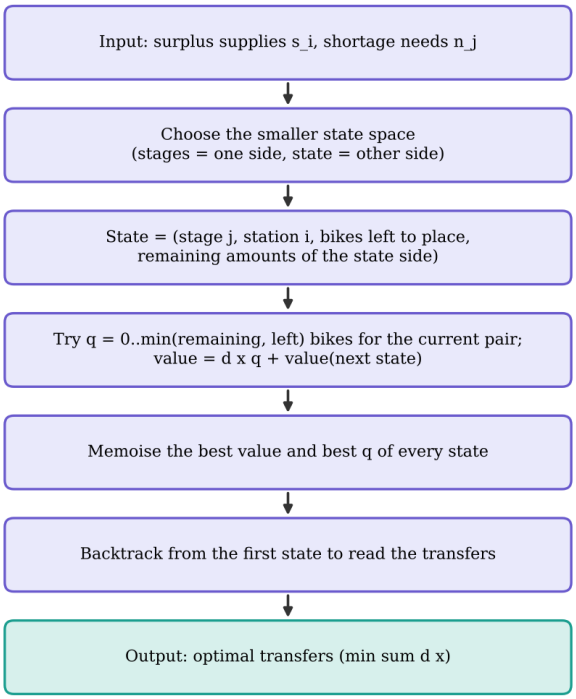


Figure 3.3: Dynamic-programming algorithm workflow

3.10 Route Planning and Constraint Validation

The route planner receives transfers and the fleet. A transfer larger than the truck capacity is split into several trips. Each trip means: drive to the source, pick up the bikes, drive to the destination, deliver them. Trips are ordered by the distance of their pickup station from the depot, and each is assigned to a truck. With the *spread work across trucks* option enabled, a trip goes to the truck whose completed route (returning to the depot) would be shortest, which distributes work over the fleet. With the option disabled, it goes to the truck whose route grows the least, which minimizes total distance and often uses a single truck. Every truck route starts and ends at the depot.

After planning, the routes are *executed* on a copy of the inventory. Execution raises an error if a pickup would take more bikes than a station holds, a delivery would exceed a station's capacity, a truck would exceed its capacity, a truck would deliver bikes it does not carry, a truck would finish with bikes on board, or more trucks than available were used. Because the check is performed on the final plan itself, it protects against faults in any optimizer.

3.11 Objective Function

Both algorithms are scored by the same expression, score = $\alpha \times D + \beta \times I_after + \gamma \times V$, where D is the total truck distance in kilometres, I_after is the remaining imbalance in bikes and V is the number of trucks dispatched. The default weights are $\alpha = 1$ per kilometre, $\beta = 2$ per bike still out of place and $\gamma = 5$ per truck, and can be changed in the dashboard. A lower score is better. The

weights express a policy: with these defaults, leaving one bike misplaced is as costly as driving two kilometres.

3.12 Parallel Scenario Evaluation

A scenario is described by a small object holding the number of stations, trucks, truck capacity, the random seed, the algorithm and the objective weights. A worker function rebuilds the system from the seed, runs one algorithm and returns a flat dictionary of metrics. Because scenarios depend only on their specification, no large objects are exchanged between processes and results are reproducible. The runner uses the *spawn* start method for portability and returns results in input order together with the wall-clock time. The same specifications can be executed serially for comparison.

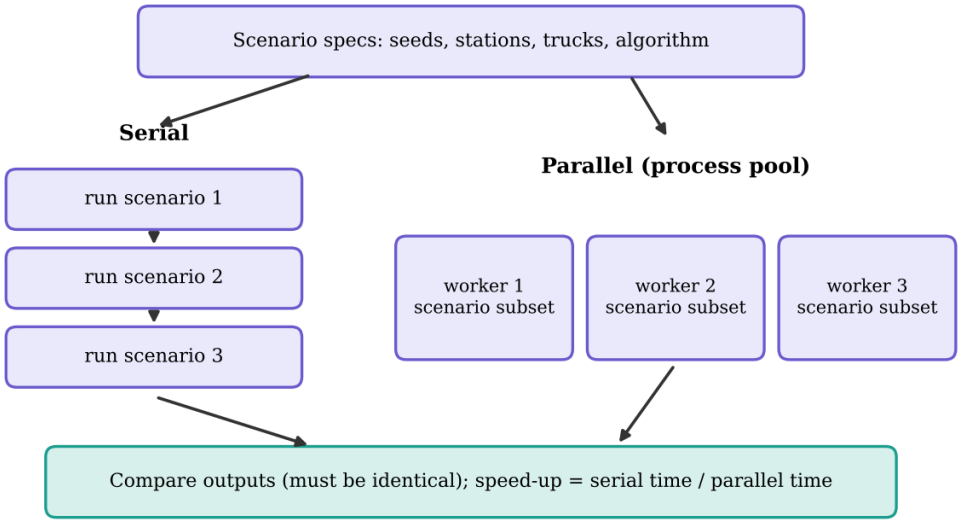


Figure 3.4: Serial and parallel scenario execution

3.13 Complexity Analysis

Module	Time	Space
Target computation	O(n log n) for rounding, O(n²) worst case for water-filling	O(n)
Greedy optimizer	O(k(m + k)) for m surplus and k shortage stations	O(m + k)
Dynamic Programming	At most (number of states) × (transitions per state); states are bounded by the product of (supply + 1) over the smaller side, so growth is exponential in the number of stations	One memo entry per state
Route planner	O(t × f) for t trips and f trucks	O(t)
Parallel runner	Work divided across workers plus start-up and communication overhead	One process per worker

Table 3.3 Complexity of the implemented modules

These are theoretical descriptions; the measured behaviour is reported in Section 5.6. The dynamic-programming time is the reason a work budget is essential: the number of states, not the number of bikes alone, determines whether a given scenario is tractable.

4. IMPLEMENTATION AND DASHBOARD DESIGN

4.1 Introduction

The application is implemented in Python with NumPy and pandas for numerical work and data handling, Streamlit and Plotly for the dashboard, and the standard multiprocessing facilities for parallel execution. The code follows the layered design of Chapter 3 and uses classes for the main entities, as listed in Table 4.3.

4.2 Project File Organization

File or directory	Purpose
app.py	Streamlit dashboard
cli.py	Command-line interface (single run or parallel batch)
models/	Station, Truck and BikeSharingSystem classes
algorithms/	Optimizer interface, Greedy and Dynamic Programming optimizers
services/	Data manager, demand analyzer, route planner, simulation engine, objective, parallel runner
visualization/charts.py	Plotly figures used by the dashboard
data/	Sample stations.csv and demand.csv
tests/	Automated tests (algorithms, constraints, parallel execution)
scripts/generate_sample_data.py	Regenerates the sample CSV files
requirements.txt	Python package dependencies

Table 4.1 Project file organization

4.3 Object-Oriented Design

Class	Responsibility
Station	Identifier, coordinates, capacity, current bikes, predicted demand and target; derived surplus, shortage and status
Truck	Capacity, current load, ordered list of pickup/delivery stops and travelled distance; enforces its own capacity
BikeSharingSystem	Holds stations, fleet size, truck capacity, depot and the distance matrix
DemandAnalyzer	Computes integer targets and the imbalance report
Optimizer (base class)	Common plan() interface returning a list of Transfer objects
GreedyOptimizer	Greedy transfer plan
DynamicProgrammingOptimizer	Exact transfer plan by memoised dynamic programming
RoutePlanner	Splits transfers into trips, assigns trucks, validates by execution
SimulationEngine	Runs optimizers on a scenario and computes metrics and objective score
ParallelRunner	Runs scenario specifications serially or on worker processes
ResultsVisualizer	Builds map, inventory, comparison and batch charts

Table 4.3 Class responsibilities

4.4 Data Management

Stations are described by a CSV file with columns station_id, lat, lon, capacity and current_bikes (and optionally name), and predicted demand by a second file with station_id and

predicted_demand. Missing demand values are replaced by the average demand. The depot is placed at the centroid of the stations. Inputs are validated: required columns must exist, station identifiers must be unique and every station's bike count must lie between zero and its capacity.

For experiments the data manager generates synthetic systems from a random seed. Stations are placed uniformly in an area of roughly eight by eight kilometres around a city-centre coordinate. Capacity is an integer between 15 and 30, the initial fill fraction follows a Beta(1.2, 1.2) distribution so that both nearly empty and nearly full stations occur, and predicted demand is uniform between 0.3 and 1.0, multiplied by a factor of 2 to 3.5 for about a quarter of the stations to represent hot spots. The same seed always gives the same system.

4.5 Target Computation Implementation

The DemandAnalyzer implements the water-filling and largest-remainder procedure of Section 3.7. It returns integer targets that satisfy $0 \leq T_i \leq C_i$ and $\Sigma T_i = \Sigma B_i$. The analyze method then returns two maps (surplus and shortage by station index) and the total imbalance, which are the inputs of the optimizers.

4.6 Greedy Optimizer Implementation

GreedyOptimizer.plan copies the surplus and shortage maps and loops as described in Section 3.8. The shortage station is selected with max over the remaining needs, ties broken by index, and the source with min over distance, ties broken by index. Each pass appends a Transfer object and removes exhausted entries. The method does not modify the system it receives.

4.7 Dynamic Programming Optimizer Implementation

The optimizer calls solve_transport(rows, columns, cost), a memoised recursive function. Its state is (row r, column c, bikes left to place for row r, tuple of remaining column amounts). For each state it tries every feasible quantity q, bounded below by what the later columns can still absorb and above by the remaining amount and the bikes left, and stores the best value and the best q in a dictionary. After solving the first state, the stored decisions are replayed to rebuild the transfer matrix. Because the search is memoised, each distinct state is evaluated once.

The wrapper DynamicProgrammingOptimizer extracts the surplus and shortage amounts, trims any excess demand defensively, builds the distance table, compares the two state-space sizes and calls solve_transport with the smaller side as the state. A RecursionError caused by a very large problem is converted into ProblemTooLargeError, so the dashboard can show a warning instead of an error.

4.8 Worked Example

Consider four stations on a straight east–west line. Stations A and B each have five surplus bikes. Station C and station D each need five bikes. The straight-line distances in kilometres are A–C 4.33, B–C 4.33, A–D 1.08 and B–D 7.59. Greedy takes the largest shortage first (C and D tie, so C is chosen by index) and serves it from the nearest surplus station; A and B are equally near, so A is chosen. D must then be served from B. Dynamic Programming evaluates both ways of pairing the stations and finds that serving C from B and D from A is much cheaper.

Plan	Transfers	Transfer cost (bike-km)
Greedy	A → C (5 bikes), B → D (5 bikes)	5 × 4.33 + 5 × 7.59 = 59.60
Dynamic Programming	B → C (5 bikes), A → D (5 bikes)	5 × 4.33 + 5 × 1.08 = 27.09

Table 4.2 Worked example: Greedy versus Dynamic Programming

The example shows why a locally best choice can be globally poor: Greedy's first, apparently harmless, choice of A consumed the only cheap source for D. The values in Table 4.2 were produced by running the project's optimizers on this layout.

4.9 Route Planner and Simulation Engine

RoutePlanner.plan splits transfers into trips of at most the truck capacity, sorts them by pickup distance from the depot and assigns each to a truck using the chosen strategy (Section 3.10). RoutePlanner.execute replays all routes on a copy of the inventory and raises ConstraintViolation for any breach. SimulationEngine.run times the optimizer, plans and executes the routes, recomputes the final imbalance from the executed inventory (rather than trusting the optimizer), and returns a RunResult containing imbalance before and after, reduction percentage, bikes moved, truck distance, transfer cost, trucks used, objective score, optimization time and total time.

4.10 Parallel Runner

ParallelRunner.run accepts a list of ScenarioSpec objects. With one worker it evaluates them in a loop. With several workers it creates a process pool using the spawn start method and maps the module-level function run_scenario over the specifications, preserving order. The function rebuilds each scenario from its seed so that serial and parallel runs perform identical work. The runner returns the result rows and the elapsed wall-clock time.

4.11 Dashboard and Command-Line Interface

The Streamlit dashboard has a configuration form and four views. The *Map & routes* view shows stations coloured as surplus, shortage or balanced (marker size indicates imbalance), one coloured route per truck, key metrics, and a table of every pickup and delivery. The *Inventory* view compares current, target and after-rebalancing bikes per station and offers a CSV download. The *Comparison* view tabulates the metrics of both algorithms and charts imbalance, distance, score and time. The *Parallel scenario batch* view runs a chosen number of seeded scenarios on a chosen number of workers, optionally repeats them with one worker to measure the speed-up, and shows averages and per-scenario charts.

Parameter	Range / options	Meaning
Dataset	Synthetic, sample CSV, uploaded CSV	Source of stations and demand
Stations (synthetic)	4 – 60	Size of the generated system
Random seed	any integer	Makes the synthetic scenario repeatable
Trucks	1 – 10	Fleet size
Truck capacity	2 – 40 bikes	Maximum load of one truck

Parameter	Range / options	Meaning
Algorithm	Greedy, Dynamic Programming, Compare Both	Optimizer(s) to run
Spread work across trucks	on / off	Route-assignment strategy
α, β, γ	non-negative numbers	Objective weights
Scenarios, workers	2 – 40, 1 – number of cores	Batch size and process count

Table 4.4 Dashboard configuration parameters

The command-line interface, cli.py, exposes the same engine: it runs one scenario with Greedy, Dynamic Programming or both, prints the metrics table and every truck route, and can run a parallel batch of seeds. It was used to produce the numerical results of Chapter 5.

5. EXPERIMENTAL SETUP, RESULTS AND DISCUSSION

5.1 Experimental Setup

All experiments use synthetic scenarios generated from fixed seeds (Section 4.4), so they can be repeated exactly. Every comparison between Greedy and Dynamic Programming uses the same scenario, the same route planner and the same objective weights ($\alpha = 1$, $\beta = 2$, $\gamma = 5$). The results were produced by running the project's command-line engine and experiment scripts; the figures in this chapter were drawn with Matplotlib from the same result data that the Streamlit dashboard displays. The dashboard itself was not used to produce the numbers, so dashboard screenshots may be inserted by the author as additional evidence.

5.2 Test Environment and Parameters

Parameter	Value	Notes
Operating system	Linux (x86-64)	Cloud sandbox used for the experiments
Python	3.13.16	NumPy 2.5, pandas 3.0
CPU cores	2	Limits the parallel speed-up
Example scenario	12 stations, 3 trucks, capacity 10, seed 42	Section 5.3
Quality experiment	10 stations, 3 trucks, capacity 10, seeds 1–30	Section 5.4
Scalability experiment	6 – 50 stations, 3 trucks, capacity 10, seeds 1–2	Section 5.5
Parallel experiment	12 stations, 3 trucks, capacity 10, seeds from 1000	Section 5.7; median of 3 repetitions
Objective weights	$\alpha = 1$, $\beta = 2$, $\gamma = 5$	Per km, per misplaced bike, per truck
DP work budget	3,000,000 transitions	Larger problems are reported as skipped

Table 5.1 Test environment and parameters

5.3 Example Scenario

The example scenario has 12 stations with a total initial imbalance of 62 bikes. Both algorithms moved 31 bikes with three trucks and reduced the imbalance to zero. Dynamic Programming achieved a lower transfer cost (76.79 versus 89.05 bike-km, 13.8% lower), a shorter total truck distance (70.64 versus 81.04 km, 12.8% shorter) and a lower objective score (85.64 versus 96.04). Its optimization time was longer, 34 ms compared with 0.3 ms, but still negligible for planning purposes.

Metric	Greedy	Dynamic Programming
Imbalance before	62	62
Imbalance after	0	0
Imbalance reduction	100%	100%
Bikes moved	31	31
Total truck distance (km)	81.04	70.64
Transfer cost (bike-km)	89.05	76.79

Metric	Greedy	Dynamic Programming
Trucks used	3	3
Objective score	96.04	85.64
Optimization time	0.30 ms	34.3 ms

Table 5.2 Example scenario: Greedy versus Dynamic Programming

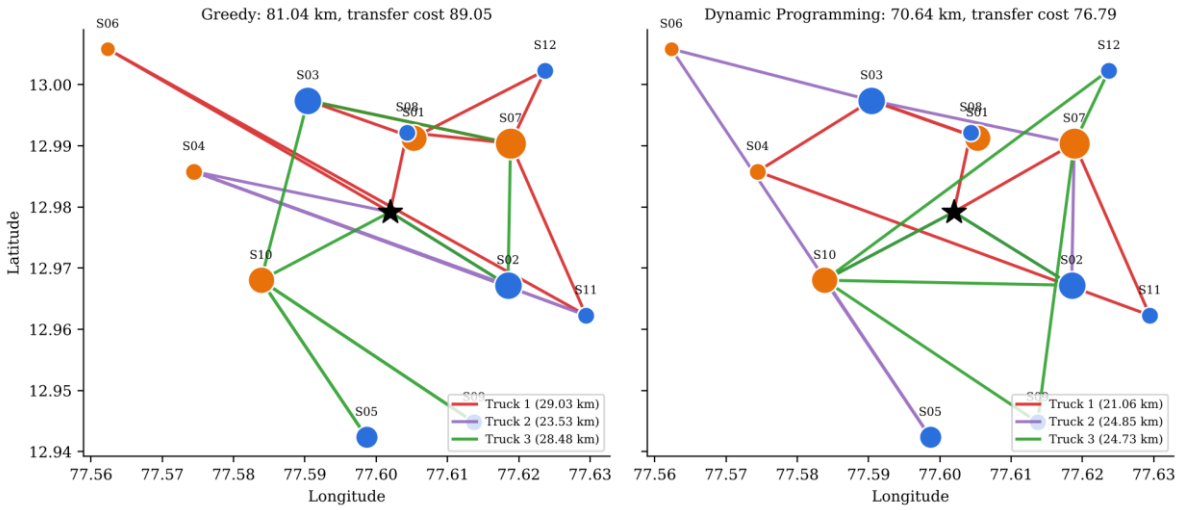


Figure 5.1: Routes planned for the example scenario

Figure 5.1 shows the routes. Blue markers are surplus stations, orange markers are shortage stations, the star is the depot, and marker size grows with the imbalance. The two plans serve the same stations but pair sources and destinations differently, which changes the tours. Table 5.3 lists the stops of each truck.

Algorithm	Truck	Route (+ pickup, − delivery)	km
Greedy	1	S08(+2) → S07(−2) → S03(+8) → S01(−8) → S12(+2) → S07(−2) → S11(+1) → S06(−1)	29.03
Greedy	2	S02(+1) → S04(−1) → S11(+1) → S04(−1)	23.53
Greedy	3	S02(+8) → S07(−8) → S03(+1) → S10(−1) → S09(+2) → S10(−2) → S05(+5) → S10(−5)	28.48
Dynamic Programming	1	S08(+2) → S01(−2) → S03(+6) → S01(−6) → S03(+2) → S04(−2) → S11(+2) → S07(−2)	21.06
Dynamic Programming	2	S02(+8) → S07(−8) → S03(+1) → S06(−1) → S05(+5) → S10(−5)	24.85
Dynamic Programming	3	S02(+1) → S10(−1) → S12(+2) → S07(−2) → S09(+2) → S10(−2)	24.73

Table 5.3 Truck routes in the example scenario

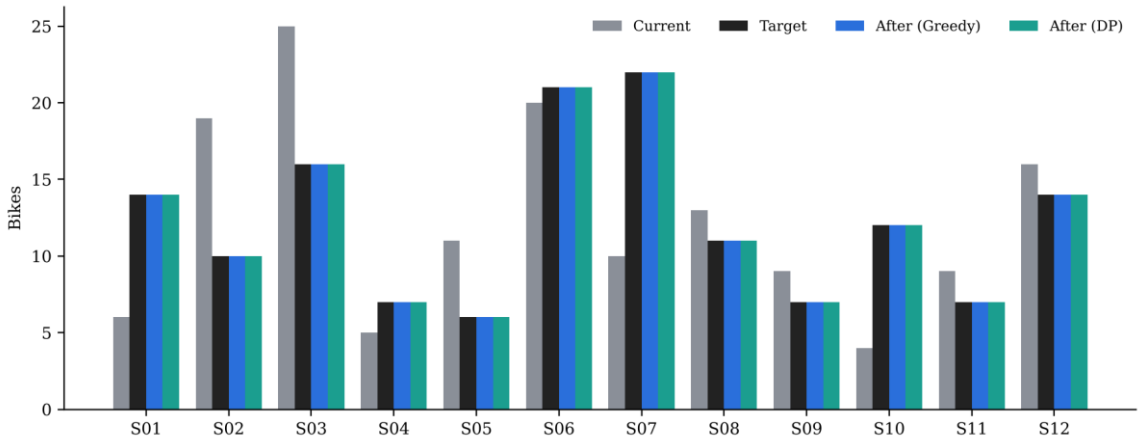


Figure 5.2: Bikes per station: current, target and after rebalancing

Figure 5.2 confirms that after rebalancing the inventory of every station equals its target for both algorithms. The independent executor recomputes the inventory from the truck stops, so this is a check of the plan itself and not only of the optimizer's claim.

5.4 Solution Quality over 30 Scenarios

To avoid drawing conclusions from one example, both algorithms were run on 30 seeded scenarios of 10 stations. All 60 runs removed the full imbalance (the mean initial imbalance was 69.9 bikes). Dynamic Programming's transfer cost was never higher than Greedy's (30 of 30 scenarios), and was strictly lower in 26 of them; in the remaining 4 the two plans cost the same. On average, Dynamic Programming reduced transfer cost by 8.2%.

Measure (mean of 30 scenarios)	Greedy	Dynamic Programming
Transfer cost (bike-km)	127.8	117.3
Total truck distance (km)	73.4	69.4
Objective score	88.4	84.4
Optimization time	0.18 ms	76.7 ms
Scenarios with lower transfer cost	–	26 of 30
Scenarios with shorter truck route	–	17 of 30

Table 5.4 Summary over 30 random scenarios

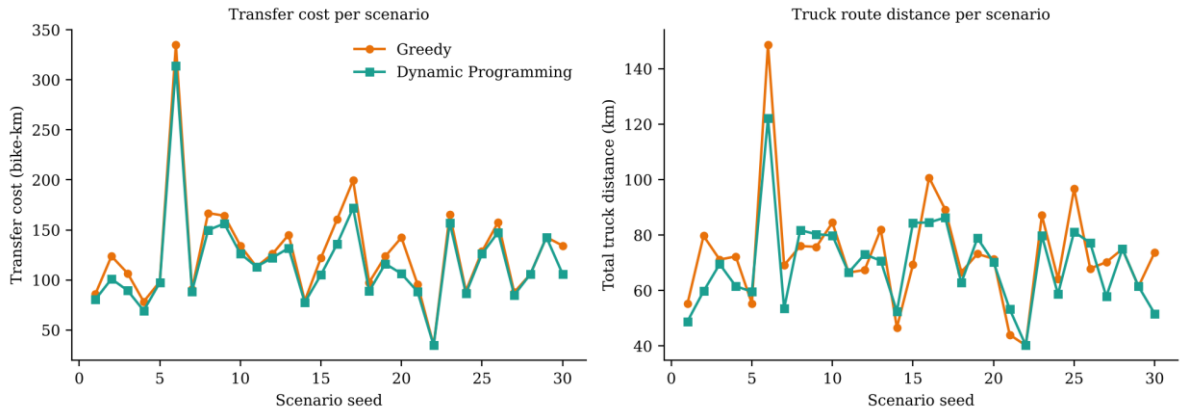


Figure 5.3: Greedy versus Dynamic Programming over 30 scenarios

The left panel of Figure 5.3 shows the guarantee provided by exact optimization: the Dynamic Programming curve never lies above the Greedy curve. The right panel shows an important subtlety. Truck distance is not the quantity Dynamic Programming minimizes, so its routes were shorter in only 17 of 30 scenarios. Averaged over all scenarios, however, the lower transfer cost translated into 5.4% shorter truck routes and a 4.5% lower objective score.

5.5 Scalability

To study growth, both algorithms were run on systems of 6 to 50 stations (two seeds each, three trucks of capacity 10). Greedy's optimization time stayed below one millisecond throughout. Dynamic Programming's time grew by roughly an order of magnitude or more for every two additional stations, from 0.16 ms at 6 stations to 0.43 s at 12 and 2.6 s at 16 stations.

Stations	Greedy time	Dynamic Programming time	DP status (2 seeds)
6	0.13 ms	0.16 ms	both completed
8	0.14 ms	1 ms	both completed
10	0.15 ms	12 ms	both completed
12	0.25 ms	433 ms	both completed
14	0.22 ms	–	both skipped
16	0.25 ms	2.64 s	both completed
18	0.32 ms	–	both skipped
20	0.28 ms	–	both skipped
30	0.49 ms	–	both skipped
50	0.76 ms	–	both skipped

Table 5.5 Optimization time versus number of stations

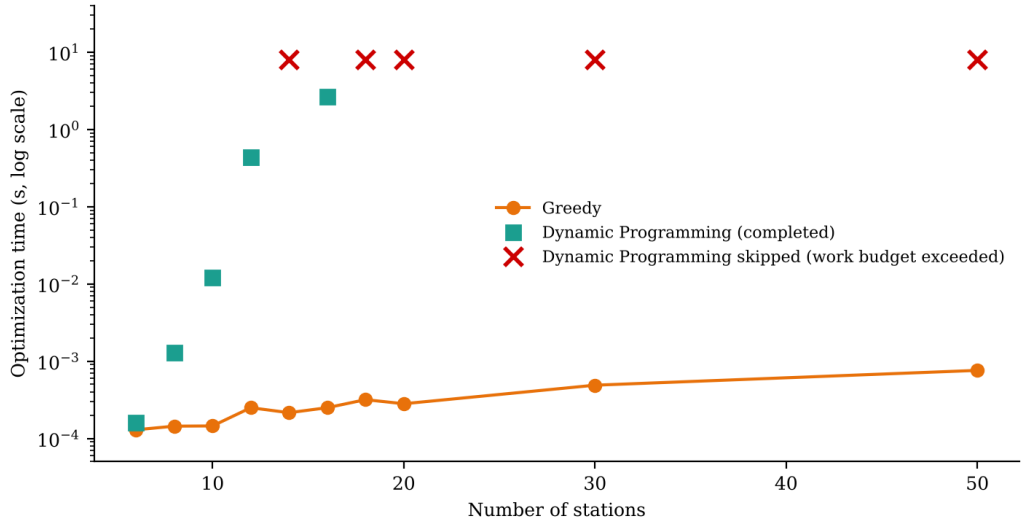


Figure 5.4: Optimization time versus number of stations

At 14 stations both runs exceeded the work budget, while both runs at 16 stations completed. This is not a contradiction: the difficulty of the dynamic program depends on how many surplus and shortage stations a scenario has and how large their amounts are, not on the number of stations alone. The measured behaviour agrees with the theory of Section 2.6: the exact method is practical

only for modest problems, and a work budget with a clear *skipped* status is necessary for a responsive tool. Where an exact answer is impractical, Greedy still gives a feasible plan instantly.

5.6 Effect of the Truck-Balancing Option

The route planner can either minimize total distance or spread work across the fleet. Using Greedy transfers on 20 scenarios of 10 stations with three trucks, the minimum-distance strategy used on average 1.0 truck and drove 73.4 km in total, so the single truck drove 73.4 km on average. The spreading strategy used 3.0 trucks and drove 76.1 km in total (3.5% more), but the longest single truck route fell to 29.1 km. The choice is therefore a trade-off between total cost and completion time, and the dashboard exposes it as a switch.

Strategy (mean of 20 scenarios)	Trucks used	Total distance (km)	Longest truck route (km)
Minimum total distance	1.0	73.4	73.4
Spread work across trucks	3.0	76.1	29.1

Table 5.6 Effect of the truck-balancing option

5.7 Parallel Scenario Evaluation

The parallel runner was tested on batches of 16, 48 and 96 runs (8, 24 and 48 scenarios, each solved by both algorithms), comparing one process with two worker processes on the two-core test machine. Each timing is the median of three repetitions. In every batch the serial and parallel results were identical (ignoring timing fields), which confirms that seeded scenarios are reproducible across processes.

Runs	1 process (s)	2 processes (s)	Speed-up	Efficiency	Results identical
16	0.30	0.65	0.46×	23%	yes
48	3.09	2.07	1.50×	75%	yes
96	15.52	8.92	1.74×	87%	yes

Table 5.7 Serial versus parallel scenario batches

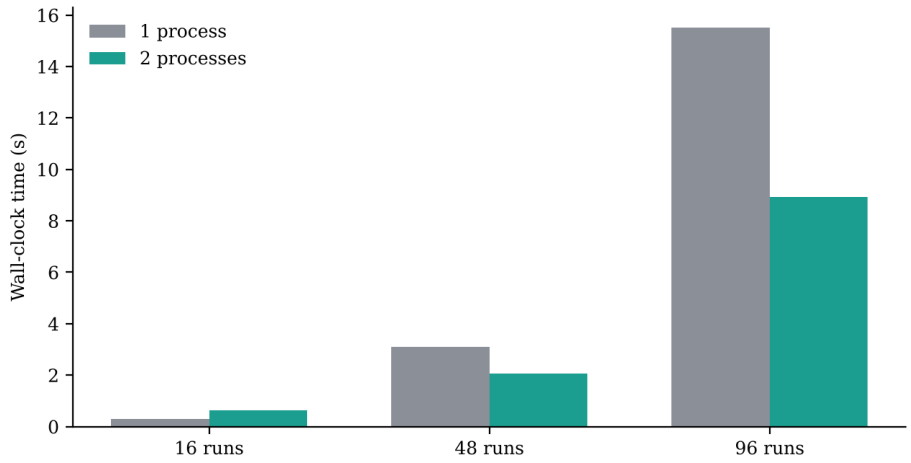


Figure 5.5: Serial versus parallel wall-clock time

For 16 runs the parallel version was slower (0.46×) because starting processes and sending work cost more than the 0.3 s of computation. For 48 runs the speed-up was 1.50× and for 96 runs 1.74×, an efficiency of about 87% of the ideal factor of two. The results support the statement of

Section 2.8 that parallelism helps only when the workload is large enough to amortize its overhead. They are measurements from one machine and should not be generalized; on a laptop with more cores a larger speed-up can be expected for large batches.

5.8 Verification by Automated Tests

The project includes 23 automated tests, all of which pass. They verify the properties required by the testing checklist of the project guide and are summarized in Table 5.8.

Group	What is checked
Targets	Targets conserve the total bikes and respect $0 \leq T \leq$ capacity for 25 random systems; a very high-demand station is capped; zero demand everywhere still works
Plan constraints	For both algorithms on random systems with 1, 3 and 5 trucks of different capacities: bikes conserved, no negative or over-capacity station, no truck over capacity, no truck ends loaded, pickups equal deliveries, fleet limit respected, and the input system is not modified
Large cases	Greedy on 80 stations; Dynamic Programming on 80 stations is skipped cleanly instead of crashing
Splitting	A transfer larger than the truck capacity is split into valid trips
Edge cases	Already balanced, empty and full stations, no surplus, no shortage, a single station
Validation	The route executor rejects over-capacity trucks, over-capacity stations and pickups from stations without enough bikes
Distances	Non-negative, symmetric, zero on the diagonal
Dynamic Programming correctness	Equal to an independent exhaustive search on 60 random small cases; never worse than Greedy on 30 scenarios; both orientations of the state-space choice give the optimum
Objective and metrics	Both algorithms see the same initial imbalance; scores equal the weighted formula; reduction percentage is computed correctly
Parallel execution	Serial, parallel and repeated parallel runs give identical results

Table 5.8 Automated test suite

5.9 Discussion

The results answer four questions. First, both algorithms produce valid plans that remove the complete imbalance, and the validation layer guarantees that every constraint holds. Second, exact optimization gives a lower transfer cost in most scenarios and never a higher one, with a measurable but modest benefit (about 8% on average here). Third, that benefit costs computation time that grows exponentially with problem size, so Greedy remains the practical choice for large systems and for interactive use. Fourth, parallel execution is useful for batches of scenarios but only when each batch is large enough to repay process overhead.

Several limits apply. All data are synthetic, so the results illustrate the methods and do not describe a real city. Distances are straight-line distances, and the cost model counts kilometres only. Dynamic Programming optimizes the simplified transfer cost, not the full routing problem, which is why its truck routes are not always shorter. Timing values come from a single two-core machine. These limits are discussed further in Chapter 6.

6. CONCLUSION, LIMITATIONS AND FUTURE SCOPE

6.1 Conclusion

This project developed a Bike-Sharing Station Rebalancing Optimizer that computes demand-based target inventories, plans truck-based rebalancing with a Greedy algorithm and an exact Dynamic Programming algorithm, validates every plan against inventory, capacity and fleet constraints, evaluates both algorithms with one objective, and runs independent scenarios in parallel. A Streamlit dashboard and a command-line interface expose the engine, and 23 automated tests verify its behaviour.

In the experiments both algorithms removed the full imbalance in every scenario. Dynamic Programming reduced the transfer cost by 13.8% in the 12-station example and by 8.2% on average over 30 scenarios, and was never worse than Greedy on that measure. Its running time, however, grew from sub-millisecond for six stations to seconds for sixteen and exceeded the work budget for larger problems, whereas Greedy remained below one millisecond at 50 stations. Parallel execution reduced the time of a 96-run batch by a factor of 1.74 on two cores but was slower for a small batch. The project therefore illustrates the central trade-off between fast heuristics and exact methods and the conditions under which parallelism helps.

6.2 Limitations

- The model is static and single-night; it does not consider demand during rebalancing, time windows or driver working hours.
- Each truck trip carries bikes from one station to one station; multi-stop loading (several pickups before deliveries) is not modelled.
- Distances are great-circle distances, not road distances, and travel time is not modelled.
- Dynamic Programming optimizes the simplified transfer cost, not the complete routing objective, and is practical only for small and medium problems.
- Predicted demand is an input; the system does not forecast demand and has been evaluated only on synthetic data.
- Timing results come from a single two-core test machine and are not general performance claims.

6.3 Future Scope

- Use real station data and trip histories, and add a demand-forecasting model that produces the predicted demand.
- Obtain road distances and travel times from a routing service and model time windows.
- Allow trucks to visit several stations per tour and optimize the visit order, following the routing formulations of the literature [5], [6].
- Replace the exhaustive transition enumeration by a minimum-cost-flow solver or integer-programming model for large systems, and compare it with the dynamic program.
- Add improvement heuristics (such as local search) on top of Greedy plans.
- Store scenarios and results in a database such as SQLite and add confidence intervals over more seeds and machines.

7. REFERENCES

[1] P. DeMaio, “Bike-sharing: History, impacts, models of provision, and future,” Journal of Public Transportation, vol. 12, no. 4, pp. 41–56, 2009.

[2] S. Shaheen, S. Guzman, and H. Zhang, “Bikesharing in Europe, the Americas, and Asia: Past, present, and future,” Transportation Research Record, no. 2143, pp. 159–167, 2010.

[3] G. Laporte, F. Meunier, and R. Wolfler Calvo, “Shared mobility systems,” 4OR, vol. 13, no. 4, pp. 341–360, 2015.

[4] T. Raviv, M. Tzur, and I. A. Forma, “Static repositioning in a bike-sharing system: models and solution approaches,” EURO Journal on Transportation and Logistics, vol. 2, pp. 187–229, 2013.

[5] D. Chemla, F. Meunier, and R. Wolfler Calvo, “Bike sharing systems: Solving the static rebalancing problem,” Discrete Optimization, vol. 10, no. 2, pp. 120–146, 2013.

[6] M. Dell’Amico, E. Hadjicostantinou, M. Iori, and S. Novellani, “The bike sharing rebalancing problem: Mathematical formulations and benchmark instances,” Omega, vol. 45, pp. 7–19, 2014.

[7] F. L. Hitchcock, “The distribution of a product from several sources to numerous localities,” Journal of Mathematics and Physics, vol. 20, pp. 224–230, 1941.

[8] G. B. Dantzig, Linear Programming and Extensions. Princeton, NJ, USA: Princeton University Press, 1963.

[9] H. W. Kuhn, “The Hungarian method for the assignment problem,” Naval Research Logistics Quarterly, vol. 2, pp. 83–97, 1955.

[10] R. Bellman, Dynamic Programming. Princeton, NJ, USA: Princeton University Press, 1957.

[11] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[12] P. Toth and D. Vigo, Eds., The Vehicle Routing Problem. Philadelphia, PA, USA: SIAM, 2002.

[13] R. W. Sinnott, “Virtues of the Haversine,” Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[14] Python Software Foundation, “multiprocessing — Process-based parallelism,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/multiprocessing.html

[15] Python Software Foundation, “concurrent.futures — Launching parallel tasks,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/concurrent.futures.html

[16] Streamlit Inc., “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io/

[17] Plotly Technologies Inc., “Plotly Python graphing library.” [Online]. Available: https://plotly.com/python/

[18] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[19] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf., 2010, pp. 56–61.